Standard A:

- Passing test cases for task 1 is sufficient.

- What is my code logic:

  The goal of task1 is finding the most direct path, which is actually to find the least Fscore. My idea is to create a **"while not frontier.empty() "loop,** so that I can use **"PriorityQueue"** to keep getting the least Fscore node as my **"current_node"**.

  To each **"current_node"**, I think about all possible actions and its successor(also with Gscore+=1), and calculate its Hscore to get the Fscore. Before Fscore is used to prioritize, I do Pruning on Gcost , which means I check if the new path to that node is shorter than any previously path (discard it if not).

- How do you know that your solution is correct?

  I passed all hidden test and I think my my code logic is right

- Did you perform any other tests yourself? What edge cases did you consider?

  Although I didn't do any other tests, I design the code with some edge cases.

  First, I set "if (x, y) == goal" to solve the situation that the original node is the goal node.

  Second, to avoid the situation that two nodes have the same Fscore and cannot be comparable, I introduce a new variable "counter". "counter" keeps plus one in each process, which means if two paths have the same Fscore, it will choose the old one.

- What assumptions does your code make?

  The code assumes the size of grid is not too large, the computer ability is able to solve the problem, the action is always successful, the scenario won't change

- Could it be applied to solve any possible search task?

  No, the code is ideal on a very simple environment, which is not suit for real world.

Standard B:

- All tests are passed

- What ideas did you have?

My first idea is to find the nearest node “A” for the start_node , and find the nearest node for “B” then repeating. However, I realized this path may not be the best paths. So I try Minimum Span Tree, which guarantee the least paths. By the way, considering the barriers, we should use a* in task1 instead of Manhattan_distance to represent the distance between nodes. However, I find mst doesn’t for some hidden test. After my discussion with AI, I realized this may because the complexity of A* is much larger than BFS.

- Did you try anything that didn't quite work?

  I tried use A* to compute the distance ,and it looks doesn’t work good enough.

- How efficient is your heuristic computationally?

  BFS+MST guarantee the efficiency by reducing the number of nodes expanded by A*. Although the efficiency is not as good as using Manhattan_distance directly, this method has much higher accuracy than using Manhattan_distance directly. So the decrease in efficiency is worth.

- How informative is it?

  It’s high. I use BFS to calculate the real distance under barriers and I use Mst to guarantee the path distance is the least.

- Is your heuristic admissible? Why or why not?

  Yes, it is. What we use to calculate distance is the nearest distance + MST. Both of them are the least path. So any real it always not larger than h*(n)

- Declare what AI was used for in task 2.

In my first design, I tried use A* to compute the distance, however it doesn’t passed all tests. I discussed this problem with AI to compare their difference in detail and get the possible reason(A* has higher complexity). AI also mentioned I can use cache mechanism to accelerate. So I finally used BFS to calculate the distance.

Standard C:

- All tests are passed

- What ideas did you have?

  My first idea was to find all possible combination of tasks, which is definitely accurate but has very low efficiency. My second idea is to use greedy algorithm. However, it's not accurate enough. Then I realized I can use two-step greedy search, which means the goal is not only to find the nearest node, but also find the "the nearest node" of the the nearest node. In this way, the complexity increase from O(n) to O(n^2), which is acceptable, but it can recognize the cluster in some degree. For example, if A is(-2,-2), B(3,3),C(3,4), the algorithm will choose B rather than choose A. Although greedy algorithm with more than two steps will be more accuracy, the complexity also increase by multiplying "n". Since two-step greedy already thinks about the cluster, I don't choose more complex way. By the way, I decide to use brutal algorithm in some size of task(n<=5) to ensure the accuracy.

  I remain using BFS to calculate the distance(with barriers). Then do brutal algorithm if n<=5, else, do two-step greedy search.

- Did you try anything that didn't quite work?

  I first tried brutal algorithm to see how it works. The code worked, but didn't pass all tests. Since I knew there must be better way, especially in large size, I gave up brutal algorithm. I then used greedy algorithm, it is obviously not informative enough.

- What assumptions does your algorithm have?

  The computing ability decreases rapidly after complexity is larger than O(n^2). Assume the map is small so that brutal algorithm will be fast enough when tasks<=5

- Is it scalable? How do you know?

  Yes it is. We use greedy algorithm with complexity O(n^2), which means the increase in number of tasks is acceptable

- Declare what AI was used for in task 3.

  I use AI to help me design my code (brutal algorithm and greedy search) and see what will happen. It also inspire me in my final way to do two-step greedy search.

Standard D:

- Here you should describe and justify the choices you made in modelling both tasks.

In task4(moveforward), what we confirm before moving is “If agent is in right place”, ”If the direction is right”, ”if agent can move to the place in some direction” and “if there’s other robot on the way”. After the movement, we should delete the original place and add the new place.

In task4(rotate), what we confirm is “If we are in the right rotation and if we can rotate to target direction” and what we do after rotation is delete the original direction and set a new direction

In task5, we need to control more than one robot, so my idea is to add a new parameter(?robot) to represent which robot is moving now. Since each robot moves in lockstep, we need a state parameter to see if a robot have done it’s move in this step. By the way, I need find a way to solve conflict. The way is to record which place are occupied in this turn, and robot can not move to these places.

In the code, I add universal-preconditions in requirements, so I can use “forall” to check if all robots are down. And I also add equality in requirements, so I can use “equal” to check if something are same(which is given up later).

In predicates, I add (?robot) parameter, and design (robot-done ?r - robot) to see if one robot has been moved and (edge-used ?from ?to - gridsquare) to prevent edge conflict.

In moveforward and rotate (task5)： ?r is added to tell which robot is moving now in a timestep； (robot-done ?r) is added to ensure each robot won’t move more than one step in a turn; (robot-at) is used to check if target place is occupied(avoid vertex collisions); (edge-used ?from ?to) is used to avoid edge collisions,

(not (equal ?from ?to)) was planned to prevent from no moving, however, it was given up in later test, since gridsquare-connected already prevented it

In wait(task5)： It’s a new action, with parameter (?r - robot ?loc - gridsquare), and let robot to wait before it’s own turn； (robot-done ?r) means the move have been done and (robot-at ?loc) means if some placed can not be moved to in this turn.

In advance-time(task5)： It’s an action used to go next timestep, we use forall to

check if everything is finished. If everything are done, we will clear every state like (edge-used) and (robot-at)

- Are there any limitations or potential issues?

In task4, we can only do something in a single robot, and there's nothing represents "cost" and "timestamp".

- Declare what AI was used for in tasks 4 and 5.

I use AI to learn about how to understand and write PDDL. AI inspire me that I can add two new actions to help me solve the problem. AI advise me to use universal-preconditions and equality, which can simplify my code writing. AI also draft all comments.

Improvement of task5

The original model the position of robot by sequence, so the MAPF timestamp is wrong. The domain is modified, so that each robot planed the action and reserve the next location. The move will happen only when all actions are chosen.

Standard E:

- Completing task 6 is largely sufficient.

- You should include a brief description of each test case you have submitted, and how they cover, and are there other limitations or potential issues?

- The single-agent path-planning domain successfully allows for planning of tasks solvable without collisions.

  In my test, I set my original direction is north, which is wrong to see if it can rotate acurrately; square4 and square8 are occupied to see if it can work. The whole path contains both direction test and occupation test. However, only one test may not enough, maybe use random seed to make a loop to test it will be better.

- The multi-agent collision-aware path-planning domain successfully plans paths even where the planned path is not optimal in the single-agent problem.

  I set a situation that robot1 need to move from square1 to square4. However, square2 is occupied by robort2, which means robot1 must find a new way to pass square5 and square6. The planner still find a path, so it works

- The multi-agent collision-aware path-planning domain successfully avoids vertex and edge collisions even where they are likely to occur.

  What we need to do is to create the situation of two robots come to a point at the same time and two robots change their place in a same turn. I let robot1 start from square1 to square3 and robot2 start from square4 to square2。They might meet together in square or change each other's place in square2 and square3. Collision doesn't happen.

- Declare what AI was used for in task 6.

I use ai to help my set path with trap randomly. AI also help me in debug, finding some problem in my domain-multiple pddl.

Improvement of task6

Since the task 5 domain is revised, I redesign test instance of task6.

Problem1: test how robot can find the available routine when there're some obstacle.

Problem2: test Multi-agent path planning in PDDL. When two robots pass the same position, one of them must wait to avoid conflict, which means the time used will be longer than single-agent.

Problem3: test collision. Collision test is used when two robots need to switch position. The modified model will prevent from reserving the same position, and also prevent from swapping collision.

Problem4: Four robots are added to test if the model can afford movement for more than two robots, which is also used the availability of cycle movement.

Standard G:

- What properties of this domain make it suitable for solving with classical search and planning based methods?

Classical search and planning means states, actions, and goals are definite and can be describe clearly, which aligns the background in assignment. For example, we know where robot and barriers are, we know how robot moves under legal action. By the way, classical search and planning based methods performs normally better in some scale problem, which is also suitable in assignment.

- What assumptions have been made about the world to simplify it for this assignment?

Static: the grid won't change and barriers are stable.

Time is discrete: robots moves in a discrete time step

Observable: All information like direction and number of tasks are known.

Independent: All tasks are independent and will not influence each other.

Deterministic: the result of any action is definite

- How could you extend the problem in ways that would remove some of those assumptions? Identify at least two different extensions, one which would still allow the use of classical planners, and one which would require the use of a different modelling technique.

classical planners: The robots may have different ability. For example, some robot can ignore barriers but will cost more in rotate, and some robot can move faster rather than one step in a timestep.

different modelling technique: We can let some of the barriers be invisible, which mean robot can't move around them until someone hit it and the distribution of barriers follows some rules. This kind of modelling may need other techniques like reinforcement learning since the information is no more definite.